XÂY DỰNG WEBSITE BẰNG PHẦN MỀM MÃ NGUỒN MỞ · BUỔI 1/6

Mã nguồn mở, giấy phép, Composer và Git

Tài liệu tóm tắt dành cho sinh viên

Giảng viên
Nguyễn Hữu Khải

Đơn vị
Khoa CNTT – Trường Cao đẳng nghề Bách Khoa Hà Nội

MỞ ĐẦU

Mục tiêu buổi học

Kiến thức

- Khái niệm, lợi ích của phần mềm mã nguồn mở
- Phân biệt giấy phép MIT, Apache 2.0, GPL
- Vai trò của Composer và Packagist
- Các vùng làm việc và nhánh trong Git

Kỹ năng

- Chọn được giấy phép phù hợp tình huống
- Đọc hiểu file composer.json và ràng buộc phiên bản
- Xác định trình tự lệnh Git cơ bản

Năng lực tự chủ

- Tôn trọng bản quyền, tuân thủ giấy phép
- Có ý thức bảo vệ thông tin nhạy cảm (mật khẩu, khóa)

01

Phần mềm mã nguồn mở

Khái niệm · phân loại · mô hình phát triển · ưu và nhược điểm

Phần mềm mã nguồn mở là gì?

Phần mềm mã nguồn mở (Open Source Software – OSS) là phần mềm được công bố kèm mã nguồn và một giấy phép cho phép mọi người sử dụng, nghiên cứu, sửa đổi và phân phối lại.

Mã nguồn (source code)

Các dòng lệnh do lập trình viên viết (PHP, JavaScript...), con người đọc và sửa được.

Hai điều kiện cần

Công khai mã nguồn **và** có giấy phép trao quyền. Thiếu một trong hai thì chưa phải mã nguồn mở.

PHẦN 1 · PHẦN MỀM MÃ NGUỒN MỞ

Phần mềm tự do: 4 quyền tự do của FSF

Free Software Foundation do Richard Stallman lập năm 1985. “Free” nghĩa là **tự do**, không phải “miễn phí”.

0 **Tự do chạy**
Chạy chương trình cho bất kỳ mục đích nào

1 **Tự do nghiên cứu, sửa đổi**
Tìm hiểu cách chương trình hoạt động và sửa theo nhu cầu (cần có mã nguồn)

2 **Tự do phân phối lại**
Sao chép và chia sẻ bản gốc để giúp người khác

3 **Tự do phân phối bản sửa đổi**
Chia sẻ bản đã cải tiến để cộng đồng cùng hưởng lợi (cần có mã nguồn)

Phân loại phần mềm theo quyền sử dụng

Loại	Giá	Mã nguồn	Sửa / phân phối	Ví dụ
Mã nguồn mở	Thường miễn phí	Công khai	Được, theo giấy phép	Linux, PHP, Laravel, Firefox
Freeware	Miễn phí	Đóng	Không	Adobe Acrobat Reader, Chrome
Shareware	Dùng thử, sau đó trả phí	Đóng	Không	WinRAR
Độc quyền	Trả phí	Đóng	Không	Windows, MS Office, PhpStorm

Lưu ý: Mã nguồn mở không cấm bán. Có thể thu tiền dịch vụ, hỗ trợ, hosting... miễn tôn trọng giấy phép.

Ưu điểm và nhược điểm

Ưu điểm

- Chi phí bản quyền thấp hoặc bằng 0
- Minh bạch: tự kiểm tra được mã, khó cài cửa hậu
- Tuỳ biến theo nhu cầu
- Cộng đồng lớn, tài liệu phong phú
- Không phụ thuộc một nhà cung cấp
- Sửa lỗi nhanh nhờ nhiều người cùng xem mã

Nhược điểm

- Hỗ trợ không có cam kết (trừ khi mua dịch vụ)
- Chất lượng không đồng đều giữa các dự án
- Dự án có thể bị bỏ dở
- Cần tự cập nhật bản vá bảo mật
- Phải hiểu và tuân thủ giấy phép

Ví dụ: lỗ hổng **Log4Shell** (2021) trong thư viện Log4j ảnh hưởng hàng triệu hệ thống.

02

Giấy phép phần mềm mã nguồn mở

Permissive và copyleft · MIT · Apache 2.0 · GPL · tương thích giấy phép

Code không có giấy phép thì sao?

All rights reserved

Mặc định theo luật bản quyền: tác giả giữ mọi quyền.

Một kho mã công khai trên GitHub **nhưng không có file LICENSE**:

- Được xem mã
- **Không** được sao chép vào dự án của mình
- **Không** được sửa và phân phối lại

→ **Công khai ≠ mã nguồn mở.**

Hai nhóm giấy phép chính

Permissive (dễ dãi)

Gần như được làm mọi thứ, chỉ cần **giữ thông báo bản quyền** và nội dung giấy phép.

Được đưa vào sản phẩm mã đóng.

Ví dụ: MIT, BSD, Apache 2.0

Copyleft

Sản phẩm phái sinh khi phân phối **phải dùng cùng giấy phép** và công bố mã nguồn.

"Tự do được truyền lại" cho người dùng sau.

Ví dụ: GPL, LGPL, AGPL

PHẦN 2 · GIẤY PHÉP

Giấy phép MIT

ĐƯỢC

Dùng, sao chép, sửa, gộp, xuất bản, phân phối, cấp phép lại, **bán**.

ĐIỀU KIỆN DUY NHẤT

Giữ thông báo bản quyền và nội dung giấy phép trong mọi bản sao.

GIỚI HẠN

Không bảo hành; tác giả không chịu trách nhiệm.

```
MIT License

Copyright (c) Taylor Otwell

Permission is hereby granted, free of charge, to any
person obtaining a copy of this software… to deal in
the Software without restriction…

The above copyright notice and this permission notice
shall be included in all copies…
```

Dùng bởi: Laravel, React, Vue.js, Bootstrap, jQuery, Node.js.

So sánh MIT, Apache 2.0 và GPL v3

Tiêu chí	MIT	Apache 2.0	GPL v3
Dùng thương mại	Có	Có	Có
Đưa vào sản phẩm mã đóng	Có	Có	Không (khi phân phối)
Phải công bố mã nguồn	Không	Không	Có (khi phân phối)
Cấp quyền sáng chế rõ ràng	Không	Có	Có
Phải giữ thông báo bản quyền	Có	Có	Có
Nhóm	Permissive	Permissive	Copyleft mạnh

PHẦN 2 · GIẤY PHÉP

Tương thích giấy phép

Khi kết hợp mã từ nhiều nguồn, các giấy phép phải "sống chung" được với nhau.



Mũi tên: mã bên trái có thể đưa vào dự án bên phải (từ dễ dãi sang chặt chẽ)

Được: đưa thư viện MIT, Apache vào dự án GPLv3.

Không: Apache 2.0 vào GPLv2; mã GPL vào sản phẩm mã đóng đem phân phối.

03

Composer và Packagist

Quản lý thư viện PHP · composer.json · composer.lock · phiên bản ngữ nghĩa

PHẦN 3 · COMPOSER

Composer và Packagist

Composer

Công cụ dòng lệnh quản lý phụ thuộc cho PHP, ra đời năm 2012 (Nils Adermann, Jordi Boggiano), giấy phép MIT.

- Khai báo thư viện cần dùng
- Tự tải đúng phiên bản và phụ thuộc bắc cầu
- Sinh **autoload** — không cần require thủ công

Packagist.org

Kho gói (repository) mặc định của Composer, nơi đăng hàng trăm nghìn gói PHP.

- Tên gói dạng vendor/package
- Ví dụ: laravel/framework, guzzlehttp/guzzle
- Xem được giấy phép, số lượt tải, phiên bản

Kiểm tra cài đặt: composer --version

PHẦN 3 · COMPOSER

File composer.json

```
{
  "name": "laravel/laravel",
  "license": "MIT",
  "require": {
    "php": "^8.2",
    "laravel/framework": "^12.0"
  },
  "require-dev": {
    "fakerphp/faker": "^1.23"
  },
  "autoload": {
    "psr-4": { "App\\": "app/" }
  }
}
```

require — thư viện cần khi chạy thật

require-dev — chỉ cần khi phát triển, kiểm thử (Faker, PHPUnit)

autoload psr-4 — namespace App\ ánh xạ tới thư mục app/

"php": "^8.2" — yêu cầu phiên bản PHP

composer.json, composer.lock và vendor/

Thành phần	Vai trò	Ai tạo	Đưa lên Git?
composer.json	Khai báo thư viện và khoảng phiên bản chấp nhận	Lập trình viên	Có
composer.lock	Ghi chính xác phiên bản đã cài của mọi gói (kể cả bắc cầu)	Composer tự sinh	Có
vendor/	Mã nguồn các thư viện đã tải về và file autoload	Composer tự sinh	Không

Ví dụ: composer.json ghi "laravel/framework": "^12.0" → composer.lock ghi bản cụ thể đã cài, chẳng hạn v12.3.0.

Ràng buộc phiên bản trong composer.json

Viết	Nghĩa là	Ghi chú
^12.0	≥ 12.0.0 và < 13.0.0	Nhận minor, patch; chặn major
^1.2.3	≥ 1.2.3 và < 2.0.0	Kiểu phổ biến nhất
~1.2	≥ 1.2.0 và < 2.0.0	Chỉ số cuối được tăng
~1.2.3	≥ 1.2.3 và < 1.3.0	Chỉ nhận patch
1.2.3	Đúng bản 1.2.3	Cố định, ít dùng

^ (caret) là mặc định khi chạy composer require: an toàn vì không tự nhảy lên major mới.

PHẦN 3 · COMPOSER

Các lệnh Composer thường dùng

Lệnh	Tác dụng
composer init	Tạo composer.json mới (hỏi đáp từng bước)
composer require vendor/pkg	Thêm gói vào require, cài đặt, cập nhật lock
composer require --dev vendor/pkg	Thêm gói vào require-dev
composer install	Cài đúng phiên bản trong composer.lock
composer update	Tìm bản mới nhất thoả composer.json, ghi lại lock
composer remove vendor/pkg	Gỡ gói
composer licenses	Liệt kê giấy phép của mọi gói đã cài
composer create-project laravel/laravel blog	Tạo dự án Laravel mới tên blog

composer install khác composer update thế nào?

`install`

- Đọc **composer.lock**
- Cài **đúng** các phiên bản đã ghi
- Không thay đổi file lock
- Dùng khi: vừa clone dự án, triển khai lên server

→ **Cả nhóm có môi trường giống hệt nhau**

`update`

- Đọc **composer.json**
- Tìm bản **mới nhất** thoả ràng buộc
- Ghi đè composer.lock
- Dùng khi: chủ động nâng cấp thư viện

→ **Chạy có chủ đích, kiểm thử lại sau đó**

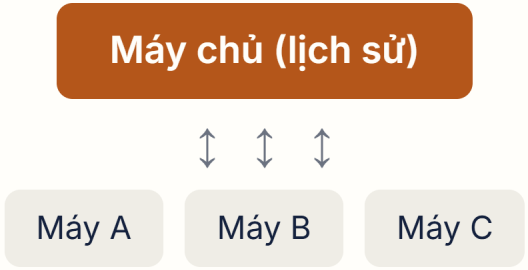
04

Hệ thống quản lý phiên bản

Khái niệm · tập trung và phân tán · Git và GitHub

VCS tập trung và VCS phân tán

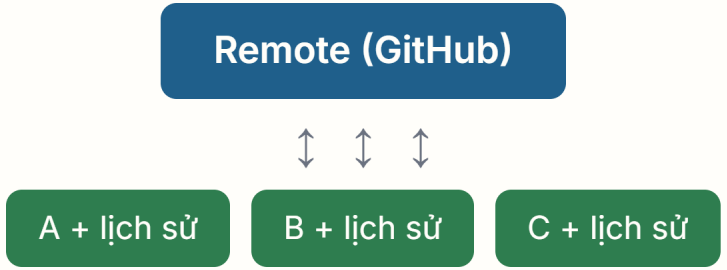
Tập trung (Centralized)



- Lịch sử chỉ nằm trên máy chủ
- Mất mạng là không commit được
- Máy chủ hỏng là mất hết

Ví dụ: SVN, CVS

Phân tán (Distributed)



- Mỗi máy có **bản sao đầy đủ** lịch sử
- Commit, xem log khi không có mạng
- Nhanh, an toàn hơn

Ví dụ: Git, Mercurial

Git khác GitHub

	Git	GitHub
Bản chất	Phần mềm quản lý phiên bản	Dịch vụ web lưu trữ kho Git
Chạy ở đâu	Trên máy tính của mình	Trên Internet (github.com)
Chủ sở hữu	Cộng đồng, mã nguồn mở	Microsoft (từ 2018)
Dùng không cần mạng	Được	Không
Chức năng thêm	—	Pull Request, Issue, Actions, Pages

Dịch vụ tương tự GitHub: **GitLab** (có bản tự cài), **Bitbucket** (Atlassian).

05

Git cơ bản

Ba vùng làm việc · lệnh cơ bản · .gitignore · nhánh · xung đột

Ba vùng làm việc của Git

Working Directory

Thư mục làm việc

Nơi em mở và sửa file bằng VS Code.

`git add` →

Staging Area

Vùng chuẩn bị (index)

Chọn những thay đổi sẽ đưa vào lần commit tới.

`git commit` →

Repository

Kho lưu trữ (.git)

Lịch sử các commit được lưu vĩnh viễn.

Ví von: **soạn đồ** (sửa file) → **cho vào thùng** (add) → **dán nhãn, niêm phong gửi đi** (commit).

Các lệnh Git cơ bản

Lệnh	Tác dụng
git init	Tạo kho Git mới trong thư mục hiện tại
git status	Xem trạng thái các file
git add <file> / git add .	Đưa file (hoặc tất cả) vào staging
git commit -m "nội dung"	Lưu staging thành một commit
git log --oneline	Xem lịch sử commit, mỗi commit một dòng
git diff	Xem thay đổi chưa add
git restore <file>	Huỷ thay đổi chưa add (quay về bản đã commit)
git restore --staged <file>	Đưa file ra khỏi staging (giữ thay đổi)

File .gitignore

```
# .gitignore của Laravel (rút gọn)
/vendor
/node_modules
/public/build
/storage/*.key
.env
.env.backup
.phpunit.result.cache
/.idea
/.vscode
```

Liệt kê những file/thư mục Git **bỏ qua**, không theo dõi.

Thư viện tải về: vendor/, node_modules/ — tạo lại được

Thông tin bí mật: .env — mật khẩu CSDL, APP_KEY, khoá API

File cá nhân, tạm: cấu hình IDE, cache, log

Làm việc với nhánh

```
# Xem các nhánh (* là nhánh hiện tại)
git branch

# Tạo nhánh mới và chuyển sang
git switch -c tim-kiem

# ... sửa code, add, commit ...

# Quay về main và gộp nhánh
git switch main
git merge tim-kiem

# Xoá nhánh đã gộp
git branch -d tim-kiem
```

git switch là lệnh mới (Git 2.23+); tài liệu cũ dùng git checkout.

Nên commit hết thay đổi **trước khi chuyển nhánh**.

Đặt tên nhánh ngắn, không dấu, nối bằng gạch ngang: them-binh-luan.

Xung đột khi gộp (merge conflict)

Hai nhánh cùng sửa **một dòng** của một file → Git không tự quyết được, đánh dấu trong file:

```
<<<<<<< HEAD
<h1>Blog của lớp CĐ</h1>
=======
<h1>Trang tin tức</h1>
>>>>>>> tim-kiem
```

Trên: bản ở nhánh hiện tại · Dưới: bản từ nhánh đang gộp vào

Cách giải quyết

1. Mở file, chọn giữ bản nào (hoặc kết hợp cả hai)
2. Xoá hết các dòng đánh dấu <<< === >>>
3. git add file đã sửa
4. git commit để hoàn tất merge

VS Code có nút "Accept Current / Incoming / Both" hỗ trợ.

TỔNG KẾT

5 điều cần nhớ hôm nay

1. Mã nguồn mở = công khai mã nguồn **+** giấy phép trao quyền. Miễn phí ≠ mã nguồn mở.

2. **Permissive** (MIT, Apache) cho đóng mã; **copyleft** (GPL) buộc mở mã khi phân phối. Laravel dùng MIT.

3. **Composer**: composer.json khai báo, composer.lock chốt phiên bản, vendor/ không commit.

4. **Git** là VCS phân tán; GitHub là dịch vụ lưu trữ kho Git.

5. Chu trình **sửa** → **add** → **commit**; dùng nhánh cho tính năng mới; **.env** luôn nằm trong .gitignore.

TỔNG KẾT

Bài tập về nhà

BÀI 1

Lập bảng so sánh 5 giấy phép **MIT, BSD, Apache 2.0, GPL v3, LGPL** theo các tiêu chí: dùng thương mại, đóng mã, công bố mã, sáng chế.

BÀI 2

Giải thích ràng buộc "laravel/framework": "^12.0". Bản 12.9.1 và 13.0.0 có được cài không? Vì sao?

BÀI 3

Viết file **.gitignore** cho một dự án PHP, giải thích từng dòng.

BÀI 4 · CHUẨN BỊ

Tạo tài khoản **GitHub**; cài Git, Composer, XAMPP; đọc trước giới thiệu Laravel tại laravel.com/docs.

Nộp bài 1–3 đầu buổi học sau (bản viết tay hoặc file).

TỔNG KẾT

Tài liệu tham khảo

Địa chỉ	Nội dung
choosealicense.com	Chọn và so sánh giấy phép mã nguồn mở
opensource.org/licenses	Danh sách giấy phép được OSI phê duyệt
getcomposer.org/doc	Tài liệu chính thức của Composer
semver.org/lang/vi	Đặc tả phiên bản ngữ nghĩa (tiếng Việt)
git-scm.com/book/vi/v2	Sách Pro Git (có bản tiếng Việt), miễn phí
laravel.com/docs	Tài liệu Laravel – chuẩn bị cho Buổi 2